# Experiment No. 9

# File Transfer Using UDP Sockets

## Aim

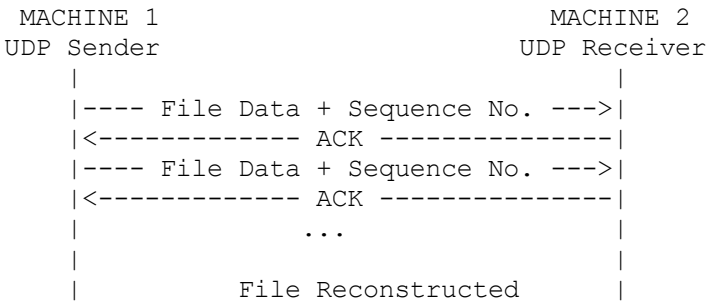
To write and execute a program using **UDP sockets** to enable file transfer between two machines over a network.

## Theory

**UDP (User Datagram Protocol)** is a connectionless transport-layer protocol. It provides a simple method for sending datagrams between two applications without establishing a dedicated connection.
For file transfer using UDP:

- The **sender/client** reads the file in small chunks.
- Each chunk is converted into a UDP datagram.
- The datagram is sent to the receiver using the receiver's **IP address and port number**.
- The **receiver/server** receives the datagrams and writes the data into a new file.
- Unlike TCP, UDP does not guarantee delivery, ordering, or duplicate protection. Therefore, for a practical implementation, the program can use **sequence numbers and acknowledgements** to improve reliability.

### Basic Working




### Important UDP Functions

| Function | Purpose |
|---|---|
| socket() | Creates a UDP socket |
| bind() | Assigns IP address and port to socket |
| sendto() | Sends UDP data |
| recvfrom() | Receives UDP data |
| close() | Closes the socket |

## Procedure

### Step 1: Prepare two machines

Connect both computers to the **same LAN/Wi-Fi network**.
For example:

- Sender IP: `192.168.1.10`
- Receiver IP: `192.168.1.20`
- Port number: `5000`

Check the receiver's IP address using:
**Windows:**
```
ipconfig
```
**Linux:**
```
ip addr
```

### Step 2: Create the receiver program

The receiver:

1. Creates a UDP socket.
2. Binds it to port `5000`.
3. Waits for the sender.
4. Receives the file name.
5. Receives file data packets.
6. Sends acknowledgement for every packet.

7. Writes received data into a file.
8. Terminates when the sender sends an `END` message.

### Step 3: Create the sender program

The sender:

1. Creates a UDP socket.
2. Specifies the receiver's IP address and port.
3. Opens the file.
4. Divides the file into smaller packets.
5. Adds a sequence number to every packet.
6. Sends each packet.
7. Waits for acknowledgement.
8. Retransmits the packet if acknowledgement is not received.
9. Sends `END` after the complete file is transferred.

### Step 4: Run the receiver first

On Machine 2:

```
python receiver.py
```

### Step 5: Run the sender

On Machine 1:

```
python sender.py sample.txt
```

Replace `192.168.1.20` in the sender program with the actual IP address of Machine 2.

---

## Program 1: UDP File Receiver

Save the following code as `receiver.py`.

```
import socketimport struct
HOST = "0.0.0.0"PORT = 5000BUFFER_SIZE = 1024HEADER_SIZE = 4
# Create UDP socketserver_socket = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
# Bind socketserver_socket.bind((HOST, PORT))
print("UDP File Receiver Started...")print("Waiting for file...")
# Receive file namedata, sender_address = server_socket.recvfrom(BUFFER_SIZE)
```

```
filename_length = struct.unpack("!I", data[:4])[0]filename = data[4:4 +
filename_length].decode()
print("Receiving file:", filename)print("Sender:", sender_address)
output_filename = "received_" + filename
with open(output_filename, "wb") as file:

    expected_sequence = 0

    while True:

        data, address = server_socket.recvfrom(BUFFER_SIZE + HEADER_SIZE)

        # Check for END message
        if data == b"END":
            server_socket.sendto(b"END_ACK", sender_address)
            break

        # Extract sequence number
        sequence_number = struct.unpack("!I", data[:HEADER_SIZE])[0]

        file_data = data[HEADER_SIZE:]

        if sequence_number == expected_sequence:
            file.write(file_data)

            # Send acknowledgement
            ack = struct.pack("!I", sequence_number)
            server_socket.sendto(ack, sender_address)

            expected_sequence += 1

        else:
            # Acknowledge duplicate packet
            ack = struct.pack("!I", expected_sequence - 1)
            server_socket.sendto(ack, sender_address)
print("File received successfully.")print("Saved as:", output_filename)
server_socket.close()
```

## Program 2: UDP File Sender

Save the following code as sender.py.

```
import socketimport structimport sys
SERVER_IP = "192.168.1.20"     # Receiver IP addressSERVER_PORT = 5000
BUFFER_SIZE = 1024HEADER_SIZE = 4TIMEOUT = 2
# Check command-line argumentif len(sys.argv) != 2:
    print("Usage: python sender.py <filename>")
    sys.exit()
filename = sys.argv[1]
# Create UDP socketclient_socket = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
# Set timeoutclient_socket.settimeout(TIMEOUT)
server_address = (SERVER_IP, SERVER_PORT)
# Send filenamefilename_bytes = filename.encode()
message = struct.pack("!I", len(filename_bytes)) + filename_bytes
client_socket.sendto(message, server_address)
print("Sending file:", filename)
sequence_number = 0
try:

    with open(filename, "rb") as file:

        while True:

            # Read file chunk
            file_data = file.read(BUFFER_SIZE)

            if not file_data:
                break

            # Add sequence number
            packet = struct.pack("!I", sequence_number) + file_data

            while True:

                # Send packet
                client_socket.sendto(packet, server_address)

                try:
                    # Wait for acknowledgement
                    ack, address = client_socket.recvfrom(HEADER_SIZE)
```

```
                ack_number = struct.unpack("!I", ack)[0]

                if ack_number == sequence_number:
                    print("Packet", sequence_number, "sent")
                    break

            except socket.timeout:
                print("Timeout. Retransmitting packet",
                      sequence_number)

        sequence_number += 1

    # Inform receiver that file transfer is complete
    while True:

        client_socket.sendto(b"END", server_address)

        try:
            response, address = client_socket.recvfrom(BUFFER_SIZE)

            if response == b"END_ACK":
                break

        except socket.timeout:
            print("Timeout. Sending END again...")

    print("File transfer completed successfully.")
except FileNotFoundError:
    print("Error: File not found.")
finally:
    client_socket.close()
```

## Execution

Suppose the two machines have the following configuration:

```
Sender Machine IP   : 192.168.1.10
Receiver Machine IP : 192.168.1.20
Port Number         : 5000
File                : sample.txt
```

### On Receiver Machine

Run:
```
python receiver.py
```
Output:
```
UDP File Receiver Started...
Waiting for file...
```

### On Sender Machine

First modify:
SERVER_IP = ″192.168.1.20″
Then run:
```
python sender.py sample.txt
```
Example output:
```
Sending file: sample.txt
Packet 0 sent
Packet 1 sent
Packet 2 sent
Packet 3 sent
File transfer completed successfully.
```
Receiver output:
```
UDP File Receiver Started...
Waiting for file...
Receiving file: sample.txt
Sender: ('192.168.1.10', 52341)
File received successfully.
Saved as: received_sample.txt
```
The received file will be saved as:
```
received_sample.txt
```

## Precautions

1. Both machines should be connected to the same network.
2. The receiver program should be started before the sender.
3. The correct receiver IP address must be specified in `sender.py`.
4. Port `5000` should not be blocked by the firewall.
5. The sender and receiver should use the same port number.
6. The original and received files should be compared to verify successful transfer.
7. UDP does not inherently guarantee delivery, so sequence numbers, acknowledgements, and retransmission are used in this implementation.

## Conclusion

Thus, the program demonstrates **file transfer between two machines using UDP sockets**. UDP provides fast and connectionless communication, but it does not inherently guarantee reliable delivery or packet ordering. The use of sequence numbers, acknowledgements, and retransmission makes the practical implementation more reliable.